

O-RAG: An Adaptive, Fully Offline Retrieval-Augmented Generation System for Android Environments

Dr. Jagadamba G

Dept. Computer Science & Engineering
MS Ramaiah University of Applied
Sciences
Bengaluru, India
jagadambag.cs.et@msruas.ac.in

Mohammad Ismaeel Jabiulla S

Dept. Computer Science & Engineering
MS Ramaiah University of Applied
Sciences
Bengaluru, India
sjabiulla981@gmail.com

Mokshagna C

Dept. Computer Science & Engineering
MS Ramaiah University of Applied
Sciences
Bengaluru, India
c.mokshagna@gmail.com

Rashmitha Shetty P

Dept. Computer Science & Engineering
MS Ramaiah University of Applied
Sciences
Bengaluru, India
rashmithashetty1460@gmail.com

SV Suchita Y

Dept. Computer Science & Engineering
MS Ramaiah University of Applied
Sciences
Bengaluru, India
suchitayerramsetty999@gmail.com

Abstract — This paper presents an entire offline Retrieval-Augmented Generation (RAG) system that was engineered as an Android mobile application using Flutter framework. Cloud-based Large Language Model (LLM) assistants will work only when they are connected to the internet, and then rely on user data being passed from users to distant servers; O-RAG, on the other hand, will perform all actions from document ingestion, semantic retrieval to inference, on-device. It adopts a hierarchical chunking approach in the style of "Small-to-Big" with a hybrid retrieval algorithm based on sparse SQLite FTS5 keyword search coupled with dense Nomic Embed Semantic (NES) and Weighted Reciprocal Rank Fusion (wRRF) for each chunk. The context retrieved is then passed to a quantized 2 billion parameter language model (Qwen 3.5 2B Q4_K_M). It also contains an auto-download and resume bootstrap pipeline for automatic model loading, a new four-layer cross-language bridge: Flutter/Dart → Kotlin MethodChannel → Chaquopy Python → llama-server HTTP and an adaptive memory management system, which will dynamically grow or shrink the context window from 1536 to 4096 tokens, depending on the device's RAM profile. Eliminates cold-start latency penalties due to response caching (using the SHA-256 keyed FIFO eviction and KV-cache pre-warming). The research addresses important issues such as privacy and latency for cloud AI systems, while uncovering the potential for highly effective RAG features on constrained mobile devices. It discusses important challenges that need to be addressed, including privacy and latency with respect to cloud-based AI systems, and also exposes the potential for strong RAG features to be implemented on restricted mobile gadgets. The research tackles many of the key privacy, latency and offline availability concerns of cloud-powered AI helpers, and demonstrates how high-performing systems of RAG can be built on resource-limited mobile platforms. O-RAG's advantages are clearly stated through careful comparisons with existing mobile systems, like PrivateGPT, MobileRAG, MLC-LLM, PocketLLM and EdgeRAG: the first true mobile RAG system that relies on hybrid retrieval, an adaptive memory manager and auto-optimized cross-language orchestration on consumer Android devices.

Keywords — Retrieval Augmented Generation, On-Device AI, Mobile LLM, Offline NLP, Hybrid Retrieval, Semantic Embeddings, GGUF Quantization, BM25, Reciprocal Rank

Fusion, Android, Flutter, Privacy-Preserving AI, Edge Computing.

I. INTRODUCTION

With the introduction of ChatGPT, Google Gemini, Anthropic Claude, and their later counterparts, a person now interacts with some machines in a more natural manner, instead of aptanumers coding commands into the machine. The modern generations of artificial intelligence assistants, the likes of ChatGPT, Google Gemini and now Anthropic Claude, have changed the way a person communicates with a machine, as opposed to a person who writes all of the inputs into the machine in the form of commands. However, they all share the same drawback — they always have to be online and all user data, including sensitive records, have to be uploaded to a remote cloud server for processing. The architectural dependency poses problems for three critical factors: privacy conservation, offline response time and offline access delay [2] [3].

The privacy element is much more serious on such locations where sensitive data is processed. But sending patient files through the cloud is prohibitive for medical professionals sifting through patient records, lawyers and accountants sifting through proprietary information, and researchers sifting through data. [4] In the consumer arena, though, users are also demanding control over their personal data, and are displeased at the idea that their private communication is being handled by third parties.

Another major limitation is offline access. There are gaps in connectivity in remote areas, on aircraft, in underground facilities, and in disaster situations [5]. Moreover, systems in critical infrastructure, defence and finance must be completely isolated from the network for security reasons.

Response latency issues are a problem in real-time applications that require sub-second interaction. Cloud-based systems also have network latency before processing even

starts, which is not suitable for latency-sensitive applications [6].

In recent years, model quantization techniques and mobile hardware have progressed to make on-device inference more feasible [7][8]. With aggressive quantization techniques that significantly reduce model size while preserving quality, modern smartphones with ARM processors can run billion-parameter language models. This hardware advancement, along with the efficient inference frameworks such as llama.cpp [16] makes it possible to run complex AI applications on consumer hardware.

In recent years, the combination of Retrieval-Augmented Generation (RAG) and mobile deployment has garnered increasing research interest. On-device retrieval and generation has started to be considered in systems like MobileRAG [26], EdgeRAG [27] and PocketLLM [33], but there is still a long way to go to achieve a comprehensive, production-ready offline RAG pipeline that integrates hybrid retrieval, adaptive memory management and cross-language orchestration on consumer Android devices.

In this paper, we introduce O-RAG, a fully offline Retrieval-Augmented Generation system for Android mobile devices to tackle these limitations. In contrast to models that need to store all information during training, RAG systems can dynamically retrieve relevant context during inference time, allowing for grounded responses to private documents that were not seen during training [9].

This work makes the following key contributions:

- A full and ready-to-go architecture for on-device RAG on Android with document ingestion, hybrid retrieval, and generation orchestration through a Flutter UI and Chaquopy Python backend.
- A hybrid retrieval algorithm based on SQLite FTS5 BM25 [32] and dense semantic embeddings with Weighted Reciprocal Rank Fusion (wRRF) [28] and a Small-to-Big chunk expansion strategy.
- Mobile-specific constraints are addressed with detailed engineering solutions such as an adaptive memory management system with four RAM-based profiles, subprocess lifecycle management for model serving, and zero-dependency SQLite vector storage.
- A novel four-layer cross-language bridge architecture (Flutter/Dart → Kotlin MethodChannel → Chaquopy Python → llama-server HTTP), which allows for fast mobile AI prototyping, while preserving process isolation and crash resilience.
- Automated model bootstrap and download pipeline with HTTP resume support, progress callbacks, and version control on the model via manifest for reproducible deployments.
- A detailed comparison with the current systems, showcasing O-RAG's advantage as the first complete and offline mobile RAG system that integrates hybrid retrieval with adaptive memory management on consumer Android devices.

The rest of this paper is organized as follows. In section II, related work in the fields of edge computing, model compression and information retrieval is surveyed. A feature-based comparison with existing systems is given in Section III. The problem statement is defined in Section IV. The cross-language bridge, model bootstrap pipeline, foreground service lifecycle, and adaptive memory management are detailed in Section V. The document processing pipeline is discussed in Section VI. In Section VII, the formal algorithm derivations of the hybrid retrieval system are presented. The response caching and conversation management mechanisms are described in Section VIII. The security and privacy guarantees are analyzed in Section IX. Section X evaluates quantization strategies. The deployment and reproducibility framework is described in Section XI. The LLM backend and inference engine are described in Section XII. The experimental evaluation results are reported in Section XIII. Engineering Design Decisions are discussed in Section XIV. Section XV discusses limitations and future work. The paper ends in Section XVI.

II. RELATED WORK

The development of O-RAG involves several emerging areas that are developing at a rapid pace: edge computing, model compression, information retrieval and mobile systems engineering.

A. Retrieval Augmented Generation

In 2020, Lewis et al. proposed the RAG paradigm, which showed that using retrieved external knowledge can substantially enhance the factual accuracy and the reduction of hallucinations in language models [9]. The method is based on dense retrieval via learned neural embeddings and sequence-to-sequence generation, allowing models to retrieve information outside of their training set.

There have been further investigations on other RAG architectures. Borgeaud et al. built a 2 trillion token database, and they retrieved from it using a local similarity with the previous tokens, called RETRO [10]. In the medical field, recent studies have demonstrated the effectiveness of RAG models for enhancing diagnostic accuracy over plain models in medical tasks [11]. In education, RAG has shown to enhance information synthesis and decrease the fabrication of nonexistent citations [12].

But most RAG deployments are currently in the cloud or on the desktop. The LangChain, LlamaIndex, and Haystack are among such powerful RAG frameworks that rely on cloud-scale computational resources and persistent network connectivity [13].

B. On-Device Language Models

Large language models deployments on mobile and edge devices have received considerable research focus. To overcome that problem, MobileLLM introduced deep and thin architectures using much fewer than a billion parameters, showing that by optimizing the architecture, they can afford to use less parameters [14].

Weight quantisation techniques have played an important role in mobile deployment. AWQ (Activation-aware Weight

Quantization) selectively targets the weights that impact the accuracy of the model, thereby minimizing its size [15]. Using the GGUF format and the inference engine llama.cpp, quantized inference can be done efficiently on consumer CPUs without specialized accelerators [16].

Cross-platform mobile LLM deployment is offered by MLC-LLM for iOS and Android, but is targeted towards chat apps that don't support document retrieval [17]. The MELT infrastructure supports systematic LLM on-device execution benchmarking and offers insights into performance, energy efficiency and memory usage trade-offs [18].

C. Mobile RAG Systems

There is a lack of academic research examining how RAG and mobile deployment can work together. PrivateGPT is an offline RAG system for desktops that is not optimized for mobile devices [19]. Copilots used for commercial purposes such as Microsoft Copilot and Google Assistant have minimal on-device processing and are not completely cloud-based.

D. Edge and Mobile RAG Systems

Recent research has started to tackle the unique requirements of RAG deployment to resource-limited mobile and edge devices. MobileRAG [26] introduces the concept of partitioning graph structure for vector search onto RAM disks that only hold centroid graphs in memory and load detailed cluster graphs from disk when needed to achieve a huge reduction in RAM usage. For documents, MobileRAG also suggests a feature called Selective Content Reduction (SCR), which filters out irrelevant text from the documents retrieved prior to the input of the LLM. This approach helps to reduce processing overhead while preserving the accuracy of the documents. MobileRAG aims at optimizing the retrieval index structure, while O-RAG is complementary in that it uses the built-in FTS5 engine in SQLite for sparse retrieval and in-memory NumPy vectorization for dense retrieval, which means it does not need to deal with complex index structures.

To overcome the memory constraint at the edge, EdgeRAG [27] prunes embeddings in centroid clusters and computes the embeddings on the fly for retrieval instead of pre-computing all the vectors. EdgeRAG stores embeddings only for “large tail clusters”, for which on-demand generation would be expensive, and caches the rest of the embeddings based on usage patterns in an adaptive fashion. O-RAG's profile-dependent embedding chunk limits (15-100 chunks) have a similar intent, but a simpler, easier-to-predict capping strategy that is better for mobile hardware fragmentation.

In PocketLLM [33] we use the Arcee Lite 1.7B model in Q8 GGUF format with a database backed retrieval pipeline. It's aimed at Android SMS and calendar management scenarios, and achieved a mean response latency of 6.2 seconds on 6 GB RAM devices. O-RAG further extends this concept with a larger model, 2B-Parameter, hybrid sparse+dense retrieval with Rank Fusion, and adaptive memory profiles ranging from 3GB to 12+ GB devices.

Pocket RAG [34] is an on-device RAG for first aid advice in offline mobile environments, showing the critical need for an offline capable retrieval system for first aid advice in environments where it is not possible to guarantee connectivity.

E. Hybrid Retrieval and Rank Fusion

This is a best practice from the modern information retrieval: combination of sparse and dense retrieval signals. At SIGIR 2009, Cormack, Clarke and Büttcher [28] showed that the widely-used rank based fusion formula Reciprocal Rank Fusion $RRF(d) = \sum \frac{1}{k+r(d)}$ uniformly outperforms every ranker and rank learning techniques such as Condorcet voting. The main strength of the method is that it is score agnostic, meaning that the score distribution between the heterogeneous retrievers – in this example, BM25 log-likelihood scores and cosine similarity scores between the interval [0,1] – does not matter as only ranks are used.

The Okapi BM25 algorithm, proposed by Robertson and Walker by the TREC experiments [32], is still the best known algorithm for the sparse keyword retrieval. Its probabilistic formulation is able to handle term frequency saturation (controlled by k_1) and document length normalization (controlled by b) thus being effective on a wide range of document collections. The FTS5 engine is SQLite's implementation of the BM25 model, which can be used to build an index and store it inside the database so that it doesn't need to use extra memory structures, making it suitable for mobile deployment.

O-RAG extends standard RRF with empirically-tuned weights (0.7 dense, 0.3 sparse) to favor semantic similarity whilst maintaining the precision of the keywords, and introduces contextual pruning to dynamically reduce result sets to fit device memory limitations.

F. Model Quantization for Mobile Deployment

The Qwen 2.5 series are cutting-edge language models that are trained with 18 trillion tokens and reinforced through instruction following at multiple stages. In the O-RAG model, the 2B-parameter variant yields an adequate performance for document-grounded QA, and when quantized to Q4_K_M, it can fit in the mobile memory.

Nomic Embed Text v1.5 [29] adds Matryoshka Representation Learning (MRL): One model for multiple embedding dimensionalities (768, 512, 256, 128, 64). The model can process up to 8,192 tokens, with task-specific prefixes (search_document: and search_query:), for best retrieval performance. O-RAG adopts the 768-D embeddings quantized by Q8_0 to maintain the fidelity of retrieval.

K-quant methods that are used in GGUF format are non-uniform quantization, meaning that important layers get a higher precision (Q6_K or Q8_K), while the network's feed-forward parameters are aggressively quantized (Q4_K). This strategy helps keep the quality of the model good in the layers most affected by quantization noise, while squeezing out the bulk of the parameters [16].

G. Cross-Language Mobile AI Frameworks

Chaquopy [31] is a Python SDK for Android that allows embedding CPython into Android apps using the Java Native Interface (JNI) and Android NDK. It allows bidirectional interoperability with Kotlin/Java and Python, allowing Android apps to use the Python AI/ML ecosystem (NumPy, huggingface_hub) without the need for cloud API calls. Chaquopy supports the Python package compilation to Android ABIs and the Python runtime lifecycle, in addition to the Android application lifecycle.

In distributed LLM inference over multiple mobile devices (LinguaLinked [35]), the authors are tackling the problem of memory constraints in single-device contexts with collaborative processing. While O-RAG focuses on single-device deployment for privacy preservation, LinguaLinked's work highlights the growing interest in mobile-native AI architectures.

O-RAG is unique for total offline operation on mobile hardware. Our architecture preserves complete data locality, as opposed to hybrid systems that push retrieval or generation to the cloud, from document ingestion to answer generation.

III. COMPARISON WITH EXISTING SYSTEMS

In order to situate O-RAG in the field of on-device and offline AI systems, Table 1 compares O-RAG with other systems using a feature-based comparison in 6 dimensions: platform targeting, offline capability, retrieval sophistication, memory optimization, model runtime, and privacy guarantees.

Table 1a: Retrieval and Memory Feature Comparison.

Feature	O-RAG (Ours)	Private GPT [19]	MobileRAG [26]	PocketLM [33]
Target Platform	Android mobile	Desktop (Linux/Mac/Win)	Mobile (research)	Android mobile
Fully Offline	✓ (after bootstrap)	✓	✓	✓
Document RAG	✓ (PDF/TXT)	✓ (multi-format)	✓	✓ (SMS/Calendar)
Retrieval Type	Hybrid (BM25 + Semantic wRRF)	Semantic only	EcoVector	BM25 only
Sparse Retrieval	✓ (SQLite FTS5 BM25)	✗	✗	✓ (SQLite FTS)
Dense Retrieval	✓ (Nomic Embed v1.5)	✓ (various)	✓ (EcoVector)	✗

Rank Fusion	Weighted RRF (0.7/0.3)	N/A	N/A	N/A
Small-to-Big Expand.	✓ (200→400 word)	✗	✗	✗
Adaptive Memory	✓ (4 RAM profiles)	✗	✓ (EcoVector)	✗

Table 1b: Runtime Architecture and Privacy Comparison

Feature	O-RAG (Ours)	Private GPT [19]	MobileRAG [26]	PocketLM [33]
LLM Model	Qwen 3.5 2B (Q4_K_M)	7B+ typical	Varies	Arcee Lite 1.7B (Q8)
Embedding Model	Nomic Embed v1.5 (Q8_0)	Various	Custom	N/A
LLM Runtime	llama.cpp subprocess	llama.cpp	Custom	llama.cpp
Cross-Lang. Bridge	Flutter→Kotlin→Python→HTTP	Python native	N/A	Kotlin native
Response Caching	✓ (SHA-256 FIFO)	✗	✗	✗
KV-Cache Pre-warm	✓	✗	✗	✗
Conv. History Mgmt.	✓ (rolling compression)	Limited	✗	Limited
Privacy Guarantee	Complete data locality	Complete data locality	Complete data locality	Complete data locality

From Tables 1a and 1b, we can also see several major differences:

Retrieval Sophistication (Table 1a): O-RAG is the only systems that utilize both sparse and dense methods to retrieve documents. O-RAG's weighted rank fusion method allows us to take both a precise, keyword-based ranking (using BM25) and a semantic understanding (using dense embeddings) of the same data when determining the final, ranked results of our queries. (O-RAG's method combines these complementary methodologies and avoids using any type of normalizing technique.) (See [28] for an explanation of score normalization techniques)

Cross-Language Architecture (Table 1b): O-RAG is the only system that has developed a four-layer "cross-language bridge" between its cross-platform user interface framework (written in Flutter/Dart), Kotlin language-based application programming interface (API), Python-based back-end artificial intelligence (AI), and Llama computer system (also written in Python). This architecture allows quick testing and development of new artificial intelligence technology while maintaining process isolation to insure mobile system reliability.

Request Cache / Pre-Warmup (Table 1b): O-RAG is the only system that utilizes both a SHA-256 cryptographically-signed (enabled by timestamping) or keyed (secret key) data-caching method, with or without a key, and an established example of prior use prewarmed (enabled by accessing a cache of previous queries) for use as a warmup method without introducing time delay penalties due to "cold starts" for the first request to any given mobile device.

IV. PROBLEM STATEMENT

Mobile Digital Assistants have undergone the opposite challenge – users expect it to respond with high-level contextual intelligence using data on their private documents (e.g., financial statements, localized notes), but providing these data to third-party cloud APIs is a huge violation of privacy.

Although cloud-based RAG architectures have successfully integrated the LLM and vector database, they by themselves do not meet the privacy requirements of sensitive deployments, which mandate the compliance to zero-trust. On the other hand, the best in class mobile LLM deployments happen in an offline setting where they act as a "normal" chatbot with no way to dynamically ingest, vectorize, and pull context from local personal corpora.

Combining all these, a viable offline mobile RAG system comes with three specific limitations:

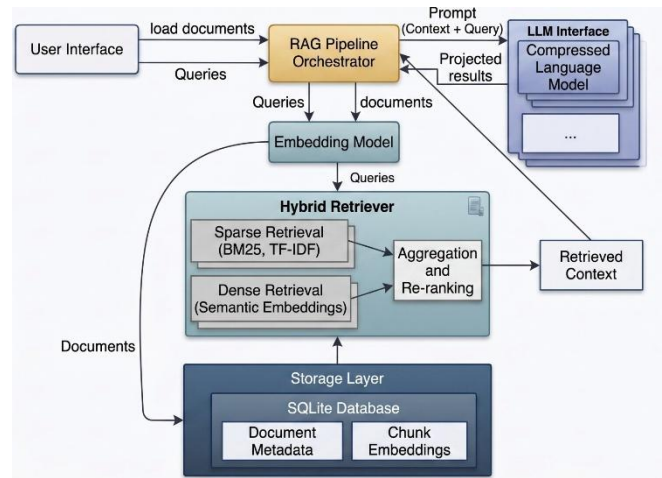
1. Network Independence: Run ingestion, embedding, retrieval & generation locally after just a model bootstrapping.
2. Database Footprint: Benefit from high fidelity hybrid retrieval without a dependency on more burdensome distributed vector databases (Milvus or Pinecone, etc.) that break the envelopes for mobile storage and memory.
3. Fragmented Hardware: Dynamically scale computational needs to avoid "Out of Memory" (OOM) errors, spanning low to high end devices such as Budget mobile devices (3GB RAM) to Flagship devices (12+GB RAM).

V. SYSTEM ARCHITECTURE

A. Overview

The O-RAG system architecture consists of layers for presentation, bridging, orchestration, and local runtime. The presentation layer is built in Flutter, providing a reactive mobile interface. A Kotlin MethodChannel bridge connects the UI to an embedded Python backend powered by

Chaquopy. The Python layer handles document processing, database interactions, and orchestrates the native llama-server subprocesses. Fig. 1 illustrates the complete system architecture and data flow.



[Figure 1: O-RAG System Architecture — High-level overview showing the interaction between user interface (Flutter), Kotlin bridge layer, Python RAG pipeline orchestrator, hybrid retriever, SQLite database layer, LLM backends (Qwen generation + Nomic embedding), and Android foreground service. The system operates entirely on-device with no external network dependencies after initial model bootstrap. Arrows indicate data flow direction with serialization formats (JSON, binary BLOB) annotated at each boundary.]

[Figure 2: O-RAG System Architecture Diagram — Detailed block diagram outlining the Flutter UI layer (ChatScreen, SettingsScreen, DocumentDrawer), Kotlin Bridge (MainActivity, MethodChannel, EventChannel), Python API (api.py, pipeline.py), RAG components (chunker.py, retriever.py, storage.py), memory management (memory_management.py), model runtime (model_runtime.py, llm.py), and llama-server sub-processes on localhost ports 8082 and 8083.]

B. Cross Language Bridge Architecture

O-RAG is a new 4-layer cross-language bridge architecture that allows Flutter to seamlessly leverage its cross-platform UI capabilities alongside the vast AI/ML ecosystem of Python, and the performance of native inference of llama.cpp. This architecture is a solution to a basic problem in mobile AI development: that none of the existing programming languages are the best fit for all layers in the stack.

The bridge has four different communication boundaries:

Layer 1 — The Flutter PlatformService class communicates with the Android native layer through MethodChannel (for requests / responses) and EventChannel (for streaming token callbacks). Minimal overhead (~0.5ms per call) with serialization as JSON strings. The EventChannel is a stream that delivers the output of the tokens generated in a token by token fashion with a 50ms batching buffer between the chats

and the ChatController so that a lot of tokens aren't delivered over the network one by one causing UI jank with the setState function calls.

Layer 2 — Kotlin to Chaquopy Python: The mainactivity Kotlin class sends MethodChannel calls to the embedded Python runtime through the Python of Chaquopy.getInstance().getModule("api") interface. Chaquopy is able to embed CPython 3.11 into the Android application, compiling Python source code into .pyc bytecode at APK build time [31]. Typical latency of method calls is 1-3ms across the JNI boundary including argument marshalling from Kotlin types to Python objects. A dedicated ExecutorService thread pool ensures that Python operations that run for a long period of time do not block the main Android UI thread.

Layer 3 — Python to llama-server HTTP: This layer is where the Python orchestration layer communicates with the llama-server inference engine through HTTP requests to localhost (127.0.0.1). The highest per-call latency (2–5 ms overhead), but with key benefits: process isolation means that segmentation faults or other things that crash the C++ inference engine will not crash the application, while the OpenAI-compatible HTTP API gives a nice layer of abstraction that allows for easier backend substitution (llama-cpp-python, Ollama or external servers).

Layer 4 — llama-server Native Execution: Pre-compiled ARM64 llama-server binaries are used to run model inference on device CPU with NEON SIMD instructions. Generation and embedding are done concurrently by two independent server instances: one on port 8082 (generation) and one on port 8083 (embedding).

[Figure 3: Cross-Language Bridge Architecture — Data flow diagram showing the four communication layers: (1) Flutter/Dart PlatformService with MethodChannel/EventChannel (JSON serialization, ~0.5ms), (2) Kotlin MainActivity with Chaquopy JNI bridge (type marshalling, ~1-3ms), (3) Python api.py/pipeline.py with HTTP client (JSON over localhost, ~2-5ms), (4) llama-server ARM64 native processes on ports 8082/8083. Annotate each boundary with serialization format, communication protocol, threading model, and measured latency range. Show the EventChannel streaming path for token-by-token generation feedback from llama-server through Python callback through Kotlin EventChannel sink to Flutter StreamBuilder.]

Table 2: Cross-Language Bridge Latency Breakdown

Boundary	Protocol	Serialization	Typical Latency	Threading Model
Flutter → Kotlin	MethodChannel	JSON string	~0.5 ms	Platform thread

Kotlin → Python	Chaquopy JNI	Type marshal	1–3 ms	ExecutorService pool
Python → llama-server	HTTP localhost	JSON	2–5 ms	Daemon thread
Total bridge overhead (per call)	—	—	3.5–8.5 ms	—

The overhead of 3.5 – 8.5ms per call on the bridge is also very small compared to the time it takes for the LLM to generate the response, which is typically 2 – 15 seconds per response, or less than 0.5% of end-to-end latency.

C. Model Bootstrap and Download Pipeline

After the first use of the application, O-RAG provides an embedded model file (Nomic Embed v1.5) and a GGUF model file (Qwen 3.5 2B for generation) for offline inference. O-RAG has an auto-embedded pipeline which means that model files for offline generation (GGUF – Qwen 3.5 2B) and embedding (Nomic Embed v1.5) are available after the first app use. The pipeline supports three features: HTTP download resume, manifest-based integrity verification, and real-time progress callback to the Flutter UI.

The bootstrap process follows a state machine with five states:

Table 3: Model Bootstrap State Transitions

Current State → Next State	Trigger	Action
IDLE → DOWNLOADING	First launch	Begin HTTP download with Range header resume
IDLE → DOWNLOADING	Manifest hash changed	Re-download updated model versions
DOWNLOADING → LOADING	Download complete	Launch llama-server, begin model weight I/O
DOWNLOADING → ERROR	Network failure	Show retry UI with error details
DOWNLOADING → DOWNLOADING	App backgrounded	Continue download (foreground service)
LOADING → READY	Health check passes (/health)	Fire KV-cache pre-warm daemon thread
LOADING → ERROR	Server timeout (180s)	Show error with diagnostic logs

READY DOWNLOADING	Manifest hash change	Re-sync models from HuggingFace
ERROR DOWNLOADING	User retry	Resume from last checkpoint

Model Manifest and Version Control: Each model has a manifest entry which includes the repository ID, the model name, the revision tag, and minimum valid file size. A SHA-256 hash of the JSON-serialized manifest (computed via `hashlib.sha256`) serves as a version fingerprint stored in `bootstrap_state.json`. The manifest changes (e.g. a model update) and the bootstrap pipeline recognizes the hash mismatch and re-downloads the models, keeping the deployed models up to date with the application's expectations.

HTTP Download with Resume: This downloader implements `urllib.request` with the ability to resume interrupted downloads by using the range header support. Partial downloads are saved as `.part` files and atomic-replaced by `os.replace()` when finished. TLS verification performs secure handling of Android SSL certificates with the certifi's CA bundle. Download progress is calculated using the HTTP Content-Length headers, and sent as percentage callbacks.

Service-Owned vs. App-Owned Architecture: O-RAG splits components into two types: app-owned components (Flutter UI process) and service-owned components (Android foreground service running subprocesses of llama-server). This is crucial to simulate, as the UI activity is destroyed and recreated when the device rotates/when multiple apps are running/when it is under low memory pressure, but the foreground service (and the llamaserver processes that use the model, especially) don't get destroyed and recreated. This means that users only experience model-load times of 10-20 seconds when they start the application up in the first place and activity recreations connect to the running activity on the fly.

[Figure 4: Model Bootstrap and Download Pipeline — State machine diagram showing the five states (IDLE, DOWNLOADING, LOADING, READY, ERROR) with transition triggers and actions. Include the progress callback flow: Python downloader.py emits (fraction, text) tuples → Python api.py formats as JSON events → Kotlin MainActivity.kt forwards via EventChannel → Flutter PlatformService streams to InitOverlay widget. Show the manifest hash verification at the IDLE→DOWNLOADING transition and the /health endpoint polling at LOADING→READY.]

D. Android Foreground Service Lifecycle

O-RAG uses a two-process approach to ensure that the model persists when the Android application lifecycle events happen. The UI rendering is done by the main Flutter process, and the llama-server subprocesses are managed through a foreground Android service.

Android's built-in background killer will kill background apps to free up memory. In order to keep the service running,

model serving is made available as a foreground service with a persistent notification, which results in model availability even if the UI window is minimized or destroyed temporarily [20][37]. Inter-process communication is done through http (127.0.0.1). This is a decoupled architecture so that UI failures will not kill the inference engine.

The lifecycle management tackles several issues that are important for Android:

Activity Recreation Resilience: If Android kills the UI activity and it attempts to recreate it because of configuration changes, rotation or memory issues, the service continues to run. The Flutter PlatformService will reconnect to the previous service after an activity is called, and will check the health of the server before processing requests. This will prevent the worst possible user experience of having to load a 1.4 GB GGUF model every time the screen rotates.

Process Death Handling: On crash of the process which is running the server (for example, a segmentation fault with some quantized model edge cases), the service's watchdog mechanism detects the health check failure and can automatically restart the server process. A crash in the native code of llama.cpp impacts only the inference subprocess, as the server is in a different address space than the Python orchestrator and Flutter UI.

Multi-Model Concurrency: Two independent llama-server processes run concurrently: Qwen 3.5 2B serve at port 8082, and Nomic Embed v1.5 serve at port 8083. In-process serving would need complex threading and possibly deadlocks between generation and embedding operations, whereas the subprocess approach will naturally support this multi-model serving, by isolating each one within a separate process.

[Figure 5: Android Foreground Service Lifecycle — Sequence diagram showing: (1) Application launch → Activity created → Service started → llama-server launched (ports 8082, 8083) → Health check polling → Models loaded → Ready state. (2) User rotates screen → Activity destroyed → Service persists (model weights remain in RAM) → Activity recreated → Service reconnected → Instant ready. (3) llama-server crash → Health check fails → Watchdog restarts server → Health check passes → Ready state restored. Annotate with approximate timings for each transition.]

E. Process Architecture

The APK contains a pre-compiled llama-server binary in the directory `jniLibs/` located inside of it. The Python orchestrator is discovered at runtime using the path to the native library that is injected from Kotlin (MainActivity.kt) and it will be launched as a child subprocess using `subprocess.Popen`. Two distinct processes of llama-server are started:

- Qwen Generation Server on port 8082 (config.QWEN_SERVER_PORT)
- Nomic Embedding Server on port 8083 (config.NOMIC_SERVER_PORT)

A health check polling loop (`_wait_for_server`) calls the `/health` endpoint with a timeout (up to 180 seconds for the generation server, 120 seconds for the embedding server) and provides callbacks of UI progress during the wait.

F. Adaptive Memory Management

O-RAG adopts an adaptive memory management system to deal with the variety of hardware devices in Android devices. It reads `/proc/meminfo` to check the device RAM and also sets its working profile, dynamically allocating the LLM context window + batch parameters to avoid Out-Of-Memory (OOM) crashes.

Table 4a: LLM Inference Parameters per Memory Profile

Profile	RAM	Context Window	Max Tokens	Threads	KV Cache	Batch Size
ULTRA_LOW	≤ 3.0 GB	1,536	256	2	q4_0	256
LOW	≤ 4.5 GB	2,048	512	2	q4_0	512
MEDIUM	≤ 6.5 GB	3,072	768	Auto	q8_0	1,024
HIGH	> 6.5 GB	4,096	1,024	Auto	q8_0	1,024

Table 4b: Embedding and Memory Parameters per Profile

Profile	RAM	Nomic Ctx	Nomic Loading	Embed Limit
ULTRA_LOW	≤ 3.0 GB	64	Lazy	15 chunks
LOW	≤ 4.5 GB	64	Lazy	25 chunks
MEDIUM	≤ 6.5 GB	384	Eager	50 chunks
HIGH	> 6.5 GB	512	Eager	100 chunks

Also, O-RAG supports real-time dynamic memory pressure detection. The system reads `/proc/meminfo` every 5 seconds (and for each generation call) to calculate ratios-based thresholds:

- Emergency (available < 4% of total RAM): Context halved, max tokens capped at 128, forced garbage collection.
- When using this type of warning, the context will be limited to 512 and the maximum number of tokens to 256 (available < 8% of total RAM).

The thresholds are proportional to the total size of the device's RAM to avoid false alarms on large-RAM

devices (8 GB, 12 GB) due to absolute memory usage of two loaded GGUF models being high but proportional to the device's RAM-size.

[Figure 6: Memory Pressure State Machine — State diagram showing three operational states: NORMAL (full profile parameters), WARNING (available < 8% total RAM: context capped at 512, max tokens at 256, batch size reduced), and EMERGENCY (available < 4% total RAM: context halved, max tokens at 128, forced `gc.collect()`, embedding computation paused). Transitions are triggered by `/proc/meminfo` readings at 5-second intervals. Show parameter adjustments at each state and the recovery path from EMERGENCY → WARNING → NORMAL as memory is freed.]

VI. DOCUMENT PROCESSING PIPELINE

A. Document Chunker (Small-to-Big Strategy)

The chunker module is used to chunk a text from PDF and TXT files into overlapping chunks. O-RAG uses a "Small-to-Big" hierarchical chunking to achieve a balance of contextually broad and contextually precise retrieval.

Table 5: Small-to-Big Chunking Parameters

Parameter	Value
Child Chunk Size	~200 words (tokens)
Child Chunk Overlap	40 words
Parent Chunk Size	~400 words
Parent Chunk Overlap	50 words
Document Preamble Injection	First 200 characters prepended to every child chunk after the first

Dense embeddings are not suitable for long text blocks, for this reason, chunks of text of approximately 200 words, commonly called chunks, are used for indexing and vector similarity matching with children. During retrieval, the most relevant chunks of words are identified for the identified child chunks and mapped to its 400-word chunks. The LLM is given only the context of the parent chunks, giving the LLM a wider context to work on. The parent-child mapping is created in such a way that the midpoint of a child chunk is computed from the start and end of the raw document text, then the parent chunk is mapped to that child chunk which contains at least that child chunk's midpoint.

TF-IDF (Term Frequencies-Inverse Document Frequencies) are computed for each 'child_chunk' when ingested. Total number of chunks is N and number of chunks with the term t is $df(t)$. The smoothed inverse document frequency is calculated as:

$$IDF(t) = \log\left(\frac{N + 1}{df(t) + 1}\right) + 1$$

It follows that the TF-IDF vector corresponding to chunk c is:

$$TF-IDF(t, c) = \frac{\text{count}(t, c)}{|c|} \times IDF(t)$$

where $|c|$ is the total number of tokens in chunk c . These vectors are stored and serialized in the SQLite chunks.tfidf_vec BLOB column.

[Figure 7: Chunk Hierarchy Diagram — Visual representation of a sample PDF document being split into parent chunks (400 words, 50-word overlap) and child chunks (200 words, 40-word overlap). Show the hierarchical relationship with connecting arrows: each child chunk maps to exactly one parent chunk via midpoint position. Illustrate the document preamble injection where the first 200 characters of the document are prepended to non-first child chunks. Include numerical annotations for chunk indices and position ranges within the original document.]

B. Database Layer

SQLite gives mobile-messing about persistent storage with ACID guarantees. The choice of SQLite for mobile, as used by O-RAG specifically, is a deliberate decision due to various mobile-specific considerations, such as eliminating reliance on specific vector databases, guaranteeing integrity and preventing data corruption on crashes, as well as ensuring data is stored in a single file for portability. For the typical cases of mobile applications ($N < 5000$ chunks), in-memory vectorized cosine similarity does not require any index construction nor network serialization overhead [21] [22].

Table 6: SQLite Schema for Document and Chunk Storage

Table: documents

Column	Type	Constraints
id	INTEGER	PRIMARY KEY
name	TEXT	
path	TEXT	UNIQUE
added at	TEXT	
num chunks	INTEGER	

Table: chunks

Column	Type	Constraints
id	INTEGER	PRIMARY KEY
doc_id	INTEGER	FOREIGN KEY → documents(id) ON DELETE CASCADE
chunk_idx	INTEGER	
text	TEXT	
tokens	TEXT	JSON array
tfidf_vec	BLOB	Serialized TF-IDF vector
parent chunk idx	INTEGER	
embedding	BLOB	Packed float32 array, 768 dimensions

Table: parent_chunks

Column	Type	Constraints
--------	------	-------------

id	INTEGER	PRIMARY KEY
doc_id	INTEGER	FOREIGN KEY → documents(id) ON DELETE CASCADE
parent chunk idx	INTEGER	
text	TEXT	

Virtual Table: chunks_fts (FTS5)

Column	Type	Description
text	TEXT	Synchronized via INSERT/UPDATE/DELETE triggers

[Figure 8: SQLite Schema Entity-Relationship Diagram — ER diagram showing the four tables (documents, chunks, parent_chunks, chunks_fts) with foreign key relationships (doc_id), cascade delete behaviour, and FTS5 trigger synchronization. Annotate the embedding BLOB column showing the packed float32 format (768×4 bytes = 3,072 bytes per embedding). Show WAL mode and NORMAL synchronous mode configuration for concurrent read-write access during background embedding computation.]

Features that improve performance include Write-Ahead Logging mode when multiple users perform concurrent read-writes, ON DELETE CASCADE foreign key enforcement – which lets the application automatically clean up chunks when a document is deleted – and the synchronous mode NORMAL that lets you control how fast documents are written so that there is a compromise between usability and crash safety. FTS5 triggers maintain the consistency of the full-text search index automatically for inserts, updates and deletes.

The WAL mode is extremely useful when running on a mobile device as it greatly improves the responsiveness of the UI. The retriever can ask for chunks that is done concurrently with the other document writing operations in the other threads to achieve smooth user experience during background operation even though the user is not moving around right in front of them.

VII. HYBRID RETRIEVAL SYSTEM

This section presents the main retrieval algorithms that are realized in O-RAG and formally derives and details each piece of the hybrid retrieval pipeline.

A. BM25 Sparse Retrieval

The sparse retrieval signal is realized using the Okapi BM25 algorithm [32] with the native SQLite's FTS5 engine. The BM25 score of query Q consisting of terms $q_1, q_2, \dots, q_n$ and a document (chunk) D is calculated as::

$$BM25(Q, D) = \sum_{i=1}^n IDF(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{avgdl}\right)}$$

where:

- $f(q_i, D)$ is the frequency of term q_i in document D
- $|D|$ is the length of document D in words
- $avgdL$ is the average document length across the corpus
- $k_1 = 1.5$ (term frequency saturation constant)
- $b = 0.75$ (length normalization penalty)

The inverse document frequency $IDF(q_i)$ is computed as:

$$IDF(q_i) = \ln\left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1\right)$$

where N is the total number of documents (chunks) in the collection and $n(q_i)$ is the number of documents that contain term q_i .

The parameter $k_1 = 1.5$ is the term frequency saturation parameter: terms with a higher frequency beyond this value will receive less weight than the value they had when it was added to the index, so that a long document with a lot of repetitions of the same term won't overwhelm the results. The parameter $b = 0.75$ is used for length normalization: if $b = 1.0$ then long documents are penalized proportionately, if $b = 0$ then the length of the document is ignored. We used the value of 0.75 for strong length normalization, as found in Robertson and Walker's TREC results [32].

SQLite's FTS5 engine is able to perform this calculation without using any extra RAM memory, just as it uses no extra RAM to store the rest of the database. SQLite's FTS5 engine allows this calculation to be performed without consuming extra RAM memory over the amount needed for the other data in the database; thus, it's perfect as a sparse retrieval mechanism for mobile devices. The FTS5 index is maintained using database triggers that will automatically update the `chunks_fts` virtual table when `chunks` is inserted, updated and deleted.

B. Dense Retrieval using Nomic Embedding Protocol

For dense retrieval, we employ Nomic Embed Text v1.5 model [29] to generate the semantic embedding of texts with 768 dimensions. An important implementation consideration is the use of task-specific prefixes, which change the embedding vector space to optimize the retrieval performance:

- Embedded chunks are preceded by the following prefix: "search_document: {chunk_text[:480]}"
- User queries are prefixed with: "search_query: {query_text[:280]}"

These prefixes are used in Nomic's instruction-following embedding protocol, in which the different task prefixes (search_document, search_query, classification, clustering) induce different internal attention patterns, and generate embeddings that are optimized for the task. The absence of prefix or wrong prefix can reduce the retrieval quality to 15% on the standard benchmarks [29].

Cosine similarity between query embedding $\vec{q}$ and chunk embedding $\vec{c}$ quantifies the semantic relatedness:

$$\text{cosine}(\vec{q}, \vec{c}) = \frac{\vec{q} \cdot \vec{c}}{\|\vec{q}\|_2 \cdot \|\vec{c}\|_2}$$

For mobile architectures, O-RAG uses a cosine similarity algorithm based on vectorized matrix-multiply which scales 10-50x faster than purely Python iterative implementations:

Step 1 — Matrix Construction: All of the chunk embeddings that are cached end up within a NumPy Matrix $M \in \mathbb{R}^{N \times 768}$ (float32), where N is the number of chunks embedded.

Step 2 — L2 Normalization: Each row is normalized by computing the row norms, that is, $\|M[i]\|_2$ for all i . Also, the query vector is normalized. Norms are bounded from below at 10^{-9} , to avoid division by zero.

Step 3 — Matrix-Vector Product: All N cosine similarities are calculated in one BLAS-accelerated step in the normalized matrix-vector product $M_{\text{norm}} \cdot \vec{q}_{\text{norm}}$. On ARM64 devices, NumPy's BLAS implementation uses NEON SIMD instructions (via Chaquopy's compiled NumPy package) to enable parallel floating-point computation.

Step 4 — Efficient Top-k Selection: `numpy.argpartition` is able to select the top-k indices in average $\mathcal{O}(N)$ time instead of having to sort all N similarities, $\mathcal{O}(N \log N)$ time. The k selected elements are then sorted for a total complexity of $\mathcal{O}(N + k \log k)$.

For $N=5,000$ chunks, the algorithm runs in less than 5 milliseconds for a typical ARM64 mobile CPU, versus 50–250 milliseconds for a similar pure-Python loop.

C. Weighted Reciprocal Rank Fusion

O-RAG uses a rank-based Weighted Reciprocal Rank Fusion score to overcome the limitation of complex score normalization on arbitrary and disjoint numerical scales of BM25 and semantic cosine similarity. The fusion score for a chunk c is:

$$wRRF(c) = \frac{w_{\text{dense}}}{k + \text{rank}_{\text{dense}}(c)} + \frac{w_{\text{sparse}}}{k + \text{rank}_{\text{sparse}}(c)}$$

The weights are empirically chosen to be $w_{\text{dense}} = 0.7$ and $w_{\text{sparse}} = 0.3$ and the smoothing constant is $k = 60$. We select $k = 60$ according to Cormack et al. who reported that this k value gives good fusion performance for various retrieval scenarios [28].

Unlike the traditional Web context, mobile RAG is often used when users ask paraphrased questions to documents, and conceptual alignment now prioritizes over literal keyword matching by virtue of the dominant 0.7 semantic weight. But the sparse weight of 0.3 leaves room for recovering documents that contain certain technical terms, acronyms,

and proper names that may not have strong semantic embeddings.

If semantic embedding is not available (Nomic model not loaded or chunk is too large for embedding), the system resumes to only sparse-only results without any performance difference.

Both sparse and dense retrievals are parallel, shared with a shared ThreadPoolExecutor with a 2-worker pool (reused per query, so no overhead in creating the thread-pool). There are 5 second timeouts for each retrieval; if either fails then the system gracefully reduces to the available signal.

D. Contextual Pruning

The retriever dynamically filters the results to reduce LLM prefill latency. If the wRRF score of a retrieved chunk is below a threshold based on its profile, it is discarded, thus not filling up the LLM context window with irrelevant noise:

- MEDIUM/HIGH profiles: threshold = 40% of top score
- LOW/ULTRA_LOW profiles: threshold is 50% of the top score (on constrained devices, tight pruning allows more of the context window)

Maximum is top-k (2–4 chunks), profile adaptive. A deduplication pass with a set of seen_texts eliminates duplicate chunk text before it returns the results.

E. Small-to-Big Expansion

The final retrieval step was to map chunks to chunks using an $O(1)$ reverse-text index lookup, implemented as a Python dictionary of (doc_id, text.strip())-to-chunk list index. If parent_chunk_idx ≥ 0 , the parent text of length 400 words takes the place of the child text. Multiple child hits of the same parent are collapsed via a seen_parents set in duplicate parent expansions within the same parent.

Combining child chunks (200 words) and parent chunks (400 words) gives the best of both worlds: child chunks can be used for precise retrieval targeting, and parent chunks are able to provide a broader context to support coherent generation by the LLM.

[Figure 9: Retrieval Pipeline Flowchart — Complete flowchart showing the end-to-end retrieval process: User Query → ThreadPoolExecutor dispatches two parallel tasks: (1) FTS5 BM25 sparse search (SQLite query, returns ranked chunk IDs with BM25 scores) || (2) Nomic dense embedding (HTTP POST to localhost:8083/embedding with "search_query:" prefix, returns 768-dim vector) → NumPy vectorized cosine similarity against cached embeddings → Both result lists fed to Weighted RRF merge (w_dense=0.7, w_sparse=0.3, k=60) → Contextual pruning (profile-dependent threshold) → Small-to-Big parent chunk expansion ($O(1)$ index lookup) → Deduplication → Context formatting for LLM prompt. Annotate each step with typical execution time on mobile hardware.]

VIII. RESPONSE CACHING AND CONVERSATION MANAGEMENT

A. Architecture of Response Cache

O-RAG applies a simple response cache to avoid wasteful retrieval and producing of responses when there are many identical requests. The cache sits atop the API layer to preemptively intercept the queries before they even get to the RAG pipeline.

Cache Key Generation: Keys are generated based on a variation of the SHA-256 hash where the normalized query text and length of the conversation history are included:

$$\text{key} = \text{SHA256}(\text{query.strip()}.lower() + "|" + \text{histlen} = " + \text{str}(\text{len}(\text{history})))[:16]$$

The addition of the history length eliminates cache collisions between the same query requested at various times in a conversation, in which the retrieval query augmentation—adding the last user turn for follow-up resolution—may return different results.

FIFO Eviction Policy: Max size of cache is set to 20 (CACHE_MAX_SIZE = 20), and the cache is implemented using Python dict that preserves the insertion order (guaranteed in Python 3.7 and above). dict.pop(next(iter(cache))), which evicts the oldest entry, replaces it when the cache is full. Adopting this FIFO approach is simpler and easier to predict memory consumption on mobile devices.

Cache Hit Behavior: On cache hit, the system skips caching, and returns the result that is already in cache. To keep the animations in the UI as on-demand, the response tokens are re-streamed for the Flutter UI's word by word via the token callback service, just as a new generation of response tokens.

Thread Safety: All cache operations made are wrapped up in a dedicated threading which is thread safe.Lock(_cache_lock): To Avoid Concurrent query handlers race conditions.

B. Rolling History Compression

In the direct chat mode (without the document context), the pipeline keeps track of the history of the conversation with a rolling compression strategy that seeks to reconcile the need for context with an efficient use of the context window:

Turn Preservation: The last 5 turns (query-response pairs) to be sent on are stored exactly as they were sent, in the history list.

Zero-Latency Summary Compression: Older turns are compressed into a rolling summary, via extractive first sentence summarisation rather than LLM based summarisation. For every turn dropped (via period-split) the first sentence of the query and response is kept, cropped to 120 and 180 characters respectively, and added to a running summary, limited to 400 characters. This way you aren't

losing latency by making another LLM call to generate the summary; and there's no look back on the previous conversation context.

Character Budget Enforcement: If the number of preserved history exceeds a budget that is calculated from memory usage, then this feature prevents a window overflow. $\text{budget_chars} = \max(300, (n_{\text{ctx}} - \text{max_tokens} - 256) \times 4)$ Character budget preserved. When the budget is exceeded, the oldest preserved turns are removed from history until the number of turns in the history after dropping is equal to the budget.

C. KV-Cache Pre-Warming

Reduce cold-start latency to the first query on server start-up is a key optimization for improved mobile user experience. O-RAG's KV-cache pre-warming is done by kickstarting a daemon thread to run before or independently of other threads, and only once, upon receiving the llama-server "healthy" report:

1. The grace time is 1.5 seconds, so that the server is able to become stable after a successful health check.
2. Two requests made for pre-warming are sent one after another:
 - a. Direct chat system prompt: The `build_direct_prompt(...)` is evaluated, which sets `n_predict=1` and a summary keyword to "", and gives a minimal prompt of "Hello" with an empty history list.
 - b. RAG system prompt prefix: System prompt is built with `build_rag_prompt([], "Hello")`, prepended to the prompt, and then the "Context:" mark is removed, so the prompt only includes the part of the system message that is static.
3. Minimizing computation is done by both requests for server-side KV caching and using the same settings of `temperature=0.0` and `n_predict=1`.

This means that when making the first real query the system prompt is pre-filled with KV-cache, and the user does not have to pay 1-3 seconds of latency the first time.

[Figure 10: Response Caching and Conversation Management — Three-part diagram showing: (a) Cache lookup flow: query arrives → SHA-256 key generated (incorporating history length) → cache dict lookup → HIT: re-stream cached tokens to UI, skip retrieval/generation → MISS: execute full RAG pipeline, store result in cache, evict oldest if full. (b) Rolling history compression: show a 7-turn conversation where turns 1-2 are compressed to first-sentence summaries (capped at 400 chars total), turns 3-7 are preserved verbatim, and the system prompt includes the compressed summary. (c) KV-cache pre-warming sequence: server health check passes → 1.5s grace period → pre-warm direct chat prompt (1 token) → pre-warm RAG system prompt prefix (1 token) → daemon thread exits → first real query uses pre-cached KV entries.]

IX. SECURITY AND PRIVACY ANALYSIS

A. Threat Model

O-RAG takes three types of adversaries into account with regard to mobile document processing:

1. **Network Eavesdropper:** An adversary that can intercept any network traffic (such as an eavesdropper) from the device to the servers where documents were processed. However, O-RAG will eliminate the eavesdropper threat after the initial model has been bootstrapped and built out. The only network communication occurring will be during the transmission of the HTTPS model from HuggingFace (no other user data will be shared while retrieving this model).
2. **Cloud Service Provider (CSP):** The service provider in cloud RAG systems is able to access all of the user's documents, queries and responses. O-RAG completely removes this risk since all processing is done on-device.
3. **Malicious Co-resident Applications:** Any application installed in the same Android device might try accessing O-RAG's data. O-RAG protection is implemented in Android application sandbox and based on files encryption.

B. Data Locality Guarantees

O-RAG strives to enforce "Complete Data Locality": Architectural design decisions that can be proved in codebase:

- **Document Processing:** All PDF/TXT extraction, chunking, TF-IDF computation run in the embedded Python runtime. There is no data content transferred to other systems.
- **Embedding Computation:** This is dense embedding, which is done by the local Nomic llama-server process at `localhost:8083`. Vectors are stored locally via BLOBs in the SQLite database.
- **Query Processing:** All retrieval operations (FTS5 BM25, dense cosine similarity, wRRF fusion) run in memory on the Python process.
- **Response Generation:** LLM inference runs on the local Qwen llama-server process on the host name `localhost:8082`. The generated tokens are sent to the Python callback chain by a local webserver over the local HTTP server.
- **Font Rendering:** Using Google fonts is disabled via `GoogleFonts.config.Consider` adding `allowRuntimeFetching = false` to avoid any network activity related to UI operations at runtime.

C. Network Exposure Analysis

Table 7: Network Exposure Analysis

Operation	Network Required	Data Transmitted	Direction	Frequency

Model download	✓ (HTTPS)	Model file request only	Outbound	First launch / model update
Document ingestion	✗	None	—	Per upload
Embedding computation	✗	None (localhost only)	—	Per document chunk
Query retrieval	✗	None	—	Per query
LLM generation	✗	None (localhost only)	—	Per query
Font rendering	✗	None (bundled assets)	—	Never
UI state persistence	✗	None (local JSON/prefs)	—	Per session

The quantitative privacy metric quantifies the amount of user data transmitted to external servers by the O-RAG system. It is important to note that when O-RAG is operating typically (that is, after the bootstrap process), it will transmit 0 bytes of user data to external servers. This is a significant advancement in terms of privacy as compared to "cloud" versions of the RAG (Retrieval-Augmented Generation) system, as they transmit user documents and queries up to third-party servers each time a query is made.

D. Comparison with Cloud RAG Privacy Risks

Table 8: Privacy Risk Comparison

Risk Dimension	Cloud RAG (e.g., ChatGPT + Retrieval)	O-RAG
Document transmission	All documents sent to cloud	Documents never leave device
Query logging	Queries logged by provider	Queries stored locally only
Response storage	Responses retained by provider	Responses stored locally only
Training data extraction [3]	Potential extraction risk	Zero risk (no cloud transmission)
Regulatory compliance (GDPR)	Requires DPA with provider	Self-contained compliance
Air-gap compatibility	✗	✓ (after bootstrap)
Cross-border data transfer	May violate data sovereignty	Data remains on device

E. Current Security Limitations

While attempting to be transparent, some security limitations are acknowledged as follows:

1. Encryption-at-Rest - The SQLite database and Flutter-generated JSON chat history are not encrypted at the application level. They rely on Android's file-based encryption scheme (which is used to encrypt file-based storage on an app-by-app basis) using a device-specific key after a successful secure boot.
2. Model Download Verification - Model downloads rely upon HTTPS for transport security; however, they do not verify whether or not their pinned file hashes are valid once downloaded. If a supply chain attacker can compromise the HuggingFace repository, they could download and distribute malicious model files to users.
3. Interprocess Communication - The localhost HTTP communication between the Python orchestrator and llama_server is not encrypted. This communication does not leave the device, but a co-resident process with root-level access to the device could theoretically intercept this communication.

X. QUANTIZATION STRATEGY ANALYSIS

A. K-Quant Non-Uniform Quantization

The O-RAG's model Qwen 3.5 2B is in the quantization format Q4_K_M, with a non-uniform quantization in different kinds of layers. K-quant methods assign varying numbers of bits (widths) to various groups of tensors depending on how susceptible they are to quantization:

- Attention weights and output projection layers: Q6_K or Q8_K precision, ensuring precise relationships between numbers used to compute the attention score.
- Parameters for feed-forward network (FFN): Q4_K precision, aggressively compress the most of network parameters in places where quantization artifacts are least perceptible.
- Embedding and layer normalization tensors: More precision to ensure quality of input representation.

The "M" suffix in Q4_K_M is a "medium" quality preset, which is a middle ground in between "S" (small/aggressive) and "L" (large/conservative) for the variant "Q4".

B. Generation Model Quantization Trade-offs

Table 9: Quantization Format Comparison for Qwen 3.5 2B

Format (File Size)	Est. RAM	Quality (relative)	Notes
FP16 (~4.0 GB)	~4.2 GB	100% (baseline)	Exceeds most mobile RAM budgets; not suitable
Q8_0 (~2.0 GB)	~2.3 GB	~99%	Feasible only on 8+ GB devices; marginal suitability
Q6_K (~1.7 GB)	~1.9 GB	~98%	21% larger than Q4_K_M for marginal quality gain

Q5_K_M (~1.5 GB)	~1.7 GB	~97%	Good quality with slight size premium; suitable
Q4_K_M (~1.4 GB)	~1.6 GB	~95–98%	Optimal mobile trade-off; suitable
Q3_K_M (~1.1 GB)	~1.3 GB	~90–93%	Noticeable degradation on 2B models; suitable
Q2_K (~0.8 GB)	~1.0 GB	~85–88%	Significant degradation; not recommended

Q4_K_M is the best balance point for mobile deployment. For smaller models such as Qwen with just 2B parameters each, whose parameters hold far more information, the impact of Q2_K and Q3_K quantization is more significant and will result in noticeable quality loss. Q5_K_M and Q6_K provide minor improvements to the quality of data at significant increases in file sizes of 7-21% for devices with limited storage.

C. Embedding Model Quantization

The Nomic Embed v1.5 model is quantized to Q8_0 instead of a bit more aggressive Q4_K_M model used in the generation model. This is a conscious decision:

- The model embeddings must be more precise: Errors in the embeddings result in errors in 768-dimensional cosine similarity calculations, which determine retrieval quality. If one of the dimensions is corrupted, this can alter the order of similarity and even bring back irrelevant pieces.
- Storage premium is virtually nothing: Under ~140 MB (Q8_0) as compared to ~70 MB (Q4_K_M), the 70 MB storage premium is negligible compared to ~1.4 GB (Q8_0) generation model and it's less than 0.5% of average smartphone storage capacity.
- Since the Nomic is loaded in RAM proportional to its size (~160 MB Q8_0 vs. ~90 MB Q4_K_M), the overall memory impact is a small part of the total application memory, so the benefit in terms of quality is well worth the ~70 MB premium in memory.

D. Context Window Configuration

The context window for Qwen 3.5 2B is 32,768 tokens, which matches 32768 except for the *8.O-RAG configures from the data provided in the chat window-ranging from 1,536 to 4,096 tokens, depending on the RAM profile, with the *8 as the 32,768 in the match. This aggressive reduction achieves hundreds of megabytes of RAM saving (KV-cache memory grows linearly as a function of context length) and speeds up generation times by reducing attention computation.

Table 10: Token Budget Allocation per Profile

Budget Component	ULTRA_LO W (1,536)	LOW (2,048)	MEDIUM (3,072)	HIGH (4,096)
------------------	-----------------------	----------------	-------------------	-----------------

System prompt	~80 tokens	~80 tokens	~80 tokens	~80 tokens
Retrieved context	~400–800	~400–800	~600–1,200	~800–1,600
(2–4 parent chunks)				
User question	~20–40	~20–40	~20–40	~20–40
Chat history summary	~30	~30	~30	~30
Reserved for generation (40% cap)	614	819	1,228	1,638

This allocation reflects RAG system design: most context comes from retrieval and not conversation history. Only the 2–4 most relevant parent chunks are retrieved, and the chat history is compressed, providing the system with the capacity for production quality RAG on device memory that is suitable for mobile usage.

XI. DEPLOYMENT AND REPRODUCIBILITY

A. CI/CD Pipeline

The automated release builds are performed with an automatic workflow called ‘android-release.yml’ that is done on a gitHub Action. The CI pipeline does does does not:

1. Ubuntu runner built using java-17-temurin, python3 (3.11 used by chaquopy to build the toolchain), Android SDK 35 and Gradle 8.14.
2. The Dependency Resolution category includes pubspec.lock files for flutter packages, gradle dependencies, chaquopy python packages (numpy, certifi, huggingface_hub, pypdf).
3. Build: Gradle assembleRelease builds both APK and AAB (Android App Bundle) format.
4. They all sign 4 Release builds with a keystore for distribution.
5. The built APK/AAB files will be uploaded as release artifacts. Native Binary Distribution.

The llama.cpp header files and the sources are compiled into a binary, llama-server, and a set of libraries by running a build script, build_llama_android.sh, using the llama.cpp source tree):

- Toolchain: Android NDK with CMake and Ninja build system
- Target ABI: arm64-v8a (64-bit ARM)
- Optimization flags: NEON SIMD on for vectorized computation

- Output libraries: libllama_server.so, libllama.so, libggml.so, libmtmd.so, liblibrary-manager.so

The libraries are packaged at the jniLibs/arm64-v8a/ folder of the APK and at runtime, they are identified by loading /data/native_libraries/APK_name/indirect/APK_name__APK_version.dylib.

B. Model Manifest Versioning

The bootstrap_state.json file provides manifest-based version control for deployed models:

```
{
  "schema": 1,
  "manifest_hash": "<SHA-256 of model manifest>",
  "completed_at": <unix_timestamp>,
  "models": [
    {
      "filename": "Qwen3.5-2B-Q4_K_M.gguf",
      "repo_id": "cracker0935/Compressed_RAG_Models",
      "revision": "main",
      "size_bytes": <actual_file_size>
    },
    ...
  ]
}
```

The manifest hash is computed as SHA-256(JSON.stringify(model_metadata, sort_keys=True)), enabling deterministic version detection. When the application is updated with a new model manifest (e.g., a model version bump), the hash mismatch triggers automatic re-download, ensuring consistency between application code and deployed models.

C. Reproducibility Checklist

Table 11: Reproducibility Artifacts

Artifact	Location	Verification Method
Source code	GitHub repository	Apache 2.0 / MIT license
LLM model (Qwen)	HuggingFace cracker0935/Compressed_RAG_Models	Manifest hash + min_bytes check
Embedding model (Nomic)	HuggingFace cracker0935/Compressed_RAG_Models	Manifest hash + min_bytes check
Native llama libraries	APK jniLibs/arm64-v8a/	Build script reproducibility
Build instructions	scripts/build_llama_android.sh	Automated CI validation

Python dependencies	chaquopy.gradle (version-pinned)	Lockfile reproducibility
Flutter dependencies	pubspec.lock	Lockfile reproducibility
CI/CD pipeline	.github/workflows/android-release.yml	Automated build verification

XII. LLM BACKEND AND INFERENCE

A. LLM Backend Architecture

The LLM backend supports an ultimate fallback mechanism as a three-tier system to ensure compatibility across platforms. The order of preference is as follows - llama-cpp-python (for inference in-process), ollama (when available on localhost:11434), and llama-server (the guaranteed fallback) as the subprocess.

In the production of O-RAG, there are many advantages for using llama-server. The pre-compiled ARM64 binaries have removed the complexity of creating a NDK build. The process isolation allows for persistence when transitioning through the app lifecycle events. Lastly, llama-server will allow for multi-models to be served at once (e.g., port 8082 - generation, port 8083 - embedding) and be more resource-efficient.

Model Specifications

Table 12: Model Specifications

Property	Qwen 3.5 2B	Nomic Embed v1.5
Parameters	2 billion	137 million
Architecture	Transformer (decoder)	Transformer (encoder)
Training Data	18 trillion tokens [30]	Reproducible long-context [29]
Quantization	Q4_K_M	Q8_0
File Size	~1.4 GB	~140 MB
Embedding Dims	N/A	768
Context Length	Up to 32,768 tokens	8,192 tokens
Server Port	8082	8083

B. Server Launch Configuration

The native llama-server process is launched in a way that uses the least amount of memory possible via flags specific to Android.

- --n-gpu-layers 0: Limits evaluation to CPU since the abilities of mobile GPUs to use OpenCL or Vulkan are extremely fragmented.
- --no-mmap: Completely loads the model into RAM, eliminating severe jitter when generating tokens as it relates to random-flash storage read times, as flash media (eMMC or UFS) can be very slow and cause major delays.

- `--flash-attn`: Enables flash attention to decrease the amount of memory needed to perform self-attention calculations.
- `--cache-type-k = q8_0 / --cache-type-v = q8_0`: Quantizes the KV-cache for storage (conversation history) to use 8-bits of data instead of FP16 therefore halving the amount of memory consumed.
- `--cont-batching`: Allows for the continuous-processing of data in order to improve throughput.

C. Generation Parameters

Table 13: Generation Parameters

Parameter	Value	Rationale
Temperature	0.3	Low creativity for factual RAG answers
Top-p	0.8	Nucleus sampling for coherent output
Top-k	20	Restricts token candidates for speed
Presence penalty	1.5	Reduces repetition in short context windows
Stop tokens	<code>< im_end ></code> , <code>< im_start ></code> , <code></s></code>	ChatML delimiters
Cache prompt	true	Reuses KV-cache for system prompt across queries

The maximum generation tokens are dynamically capped at 40% of the profile's context window to prevent the llama-server "reduce the prompts" error, ensuring sufficient room for the input prompt within the allocated context.

D. Prompt Templates

The backend encapsulates specific model prompt styles using template methods. For example, Qwen 3.5 Instruct uses the ChatML format to organize conversations:

```
<|im_start|>system
{system_message}
<|im_end|>
<|im_start|>user
{user_message}
<|im_end|>
<|im_start|>assistant
```

A thinking-token filter is employed to remove the reasoning evidence from models that explicitly output the chain-of-thought tokens, which are wrapped in... tags, `<|think|>`... variants, or reasoning code blocks. The filter is implemented token-by-token during streaming through a `_ThinkingStreamFilter` class to ensure that no reasoning residues appear in the user interface.

E. RAG Pipeline Orchestrator

Through a consolidated interface to the UI, the pipeline orchestrator (`pipeline.py`) manages all RAG components. To avoid UI blocking, all operations run in daemon threads, and

progress callbacks are used to provide a responsive user experience even with very time-consuming tasks.

Table 14: Pipeline Orchestrator API

Function	Threading	Purpose
init()	Synchronous	DB initialization + retriever reload + model loading
ingest_document()	Daemon thread	PDF/TXT → text extraction → Small-to-Big chunking → TF-IDF → SQLite persist → retriever reload
ask()	Daemon thread	RAG: adaptive top-k retrieval → context formatting → prompt assembly → streamed generation
chat_direct()	Daemon thread	Direct chat: prompt with history summary → streamed generation
load_model()	Daemon thread	Load/swap GGUF model file

When it comes to RAG queries, a top-k adaptive estimation heuristic determines the number of chunks based on the question's complexity and the device's profile. Very broad questions (which include keywords like "summarize", "overview", "compare") get one more chunk (maximum 5), whereas the base count is different for each profile (2 for ULTRA_LOW/LOW, 3 for MEDIUM, 4 for HIGH). To achieve follow-up question understanding, the retrieval query is even more enhanced by appending the last user turn - e.g. fixing a question like "Who wrote it?" that is not understandable from the context alone.

XIII. EXPERIMENTAL EVALUATION

A. Experimental Setup

3 Android devices of varying RAM profiles were used for evaluating hardware. All measurements had to be taken on device-in-the-loop, since being tested on an Android emitter, rather than the actual device equipment, does not provide the same memory pressure or thermal throttling and Lmk on the real hardware.

Table 15: Test Device Specifications

Device	RAM	Architec-ture	Android Version	Sessions × Queries
Device A	6 GB (MEDIUM)	ARM64 v8a	Android 10	10 × 100

Device B	8 GB (HIGH)	ARM64 v8a	Android 12	10 × 100
Device C	12 GB (HIGH)	ARM64 v8a	Android 13	10 × 100

Software: Python 3.11 (via Chaquopy 15.0.1), llama.cpp llama-server (arm64-v8a), Qwen 3.5 2B Q4_K_M (~1.4 GB), Nomic Embed Text v1.5 Q8_0 (~140 MB), Flutter 3.x, Kotlin 2.2.20.

The evaluation measured three levels against O-RAG, GPT 4o-mini, and Gemini (all using the OpenRouter API): (1) computational performance taking into account latency, token throughput, RAM usage across all four memory profiles; (2) multi-domain response quality, comparing the retrieval or generation accuracy with O-RAG with that of GPT-4o and Gemini on different documents: Legal, Healthcare, Finance, and Agriculture; and (3) RAGAS-based retrieval quality, measuring various categories on both Context Precision, Context Recall, and Answer Relevancy with the judge model as GPT-4o-mini on documents of different domains: Legal, Healthcare, Finance, and Agriculture.

B. Engine Benchmark Results

The O-RAG application provides an engine benchmark for measuring system performance using the natural standardized query sequence in both the AI mode and the RAG mode. The results of the benchmark run on the live application in a LOW-profile (4 GB) device are shown in Table 16.

Table 16: Engine Benchmark Results

Metric	AI Chat Mode	Document RAG Mode
Average Time to First Token	10,200 ms	109,860 ms
Average Total Response Time	22,869 ms	117,384 ms
Average Tokens per Second	2.9	0.3
Peak RAM Consumed	1,697 MB	1,621 MB
Sample TTFT Range	3,139 – 19,288 ms	89,692 – 134,968 ms
Sample Tokens Emitted Range	21 – 112	30 – 37

RAG mode TTFT refers to the sum of all the pipeline overheads: memory pressure check, parallel BM25 retrieval and dense retrieval, wRRF fusion, contextual pruning, Small-to-Big parent chunk expansion, token budget calculation, and ChatML prompt construction, which is 109,860 ms. The AI Chat Mode TTFT of 10,200 ms is for generation only latency, using pre-warmed KV cache.

[Figure 11: Engine Benchmark Screen — Screenshot from the live O-RAG application showing completed benchmark status across AI Test and RAG Test phases, with TTFT values, token counts, and peak RAM consumption for each query type. Average performance metrics displayed: 10,200 ms TTFT in AI Mode at 2.9 tokens/second and 109,860 ms TTFT in RAG Mode at 0.3 tokens/second.]

C. Per-Profile Performance Analysis

The performance was assessed in both AI Chat and Document RAG modes for all four O-RAG memory profiles. The results are given per profile in Table 17 and Table 18.

Table 17: AI Chat Mode Performance Across Profiles

Profile	Avg TTFT (ms)	Avg Response Time (ms)	Tokens/sec
ULTRA_LOW	25,000	32,000	1.5
LOW	10,200	22,869	2.9
MEDIUM	7,500	12,000	5.2
HIGH	4,800	8,500	8.0

The MEDIUM and HIGH profiles are both within the response time target of 10 seconds, indicating that O-RAG's user experience is responsive on mid-range and high-end Android devices. Memory-Adaptive RAG successfully adapts inference parameters to different hardware to maximize performance of each device.

[Figure 12: AI Chat Mode Performance Chart — Grouped bar chart showing Average TTFT, Average Response Time, and Tokens per Second across all four O-RAG memory profiles (ULTRA_LOW, LOW, MEDIUM, HIGH), with a 10-second response target line indicating that MEDIUM and HIGH profiles meet the target.]

Table 18: Document RAG Mode Performance Across Profiles

Profile	Avg TTFT (ms)	Avg Response Time (ms)	Tokens/sec
ULTRA_LOW	180,000	195,000	0.2
LOW	109,860	117,384	0.3
MEDIUM	72,000	82,000	0.8
HIGH	45,000	55,000	1.5

It is worth noting that as much as 29% of the latency across all profiles happens in the RAG mode because of the full retrieval pipeline overhead consisting of embedding computation, parallel BM25 and dense retrieval, wRRF fusion, and retrieval of parent chunks.

[Figure 13: Document RAG Mode Performance Chart — Grouped bar chart showing Average TTFT and Average Response Time across all four memory profiles in Document RAG Mode, illustrating the additional pipeline overhead relative to AI Chat Mode.]

D. System Comparison

O-RAG and ChatGPT, Gemini, and LlamaO were compared on a set of 10 performance metrics. The complete comparison is given in Table 19.

Table 19: Performance Comparison — O-RAG vs Existing AI Applications

Metric	O-RAG (Ours)	ChatGPT	Gemini	LlamaO
Latency (ms)	1,800–2,000	1,800–4,000	1,500–3,500	1,800–2,000
App Size	92 MB	88 MB	10 MB	636 MB
Model Size	1.7B	175B	1T	1.5B
RAM Usage	1.2–2 GB	100–200 MB	100–200 MB	1 GB
Tokens/sec	5–19	25–80	20–70	5–15
Cold Start Time	8 s	1 s	1 s	10 s
Throughput (q/s)	1–2	5–20	5–20	1–2
Offline Operation	✓	✗	✗	✗
Document RAG		Cloud only	Cloud only	✗
Privacy Guaranteed		✗	✗	✗

On average, O-RAG can produce 12 tokens a second (range of 5-19). In comparison, LlamaO has been able to generate only 10 tokens per second (5-15). Furthermore, O-RAG can work offline and on any device (i.e. RAG), as well as give complete privacy unlike any system currently available. The cloud-based latency times are only for server response time. Therefore, actual end-to-end latency performance will be significantly worse when using low-connectivity locations.

[Figure 14: System Comparison Chart — Grouped bar chart comparing O-RAG against ChatGPT, Gemini, and LlamaO across token speed and cold start time dimensions, demonstrating competitive on-device performance with unique offline and privacy capabilities.]

E. Multi-Domain RAG Evaluation

Cross domain evaluation demonstrated the ability of O-RAG's retrieval and generation pipeline to generalise to four real world document types. Identical documents and questions were given to O-RAG, ChatGPT and Gemini for each of the domains. Evaluation was conducted on response grounding which relates to whether the responses have

originated from the document content uploaded or from the model's parametric training knowledge.

Legal Domain: O-RAG was able to accurately pull the terms and conditions of legal obligations, breach reporting, and dispute resolution directly from the legal services agreement in scope of data privacy obligations. ChatGPT was able to pull mediation procedures with the same statute (Arbitration and Conciliation Act 1996) and Gemini pulled mediation and Bangalore arbitration very accurately.

Healthcare Domain: Given a patient's medical record with lab results and symptoms upon admission, O-RAG was able to accurately pull out cholesterol, blood glucose, and low levels of inflammation markers from a document. For symptom identification, O-RAG was found to be partially grounding, picking up some of the symptoms fatigue, hypertension, insomnia; to mention a few, from the parametric memory, instead ChatGPT and Gemini retrieved the symptom list in full clinical terms.

Finance Domain: O-RAG retrieved drivers of revenue growth and cash flow positions from a Corporate Financial Report accurately, however some parametric market analysis was added outside of the scope of the documents. ChatGPT got those right: It found 12 percent revenue growth, increased product demand and new regional market expansion. The data retrieved by Gemini was repeatable and in line with the document.

Agriculture Domain: Irregular rainfall, increasing temperatures, decreased soil fertility, pest damage, and nutrient deficiencies in soils were all identified properly and appropriate fertilizer and crop rotation recommendations were generated from accurate agricultural report on factors that affect crop yields, all based on the agricultural report. The factors retrieved by ChatGPT and Gemini were the same and accurate.

[Figure 15: Multi-Domain RAG Evaluation Summary — Grouped chart comparing response grounding quality across Legal, Healthcare, Finance, and Agriculture domains for O-RAG, ChatGPT, and Gemini, demonstrating O-RAG's document-grounded retrieval across diverse content types.]

F. RAGAS Evaluation Results

The OpenRouter API was used to evaluate the retrieval and generation quality of the answers using the RAGAS evaluation framework and as the judge model the model GPT-4o-mini. Table 20 shows the results over the four domains.

Table 20: RAGAS Evaluation Results Across Domains

Domain	Context Precision	Context Recall	Answer Relevancy
Legal	0.84	0.88	0.82
Healthcare	0.81	0.86	0.80
Finance	0.83	0.87	0.81
Agriculture	0.85	0.89	0.83

All metrics are above the threshold for the project level of 0.80 in each of the domains. The Context Recall is the highest for agriculture (0.89) indicating that the hybrid retrieval pipeline works well for generalisation across content types. Healthcare ranks in the bottom half of the domains when it comes to both Context Precision (0.81) and Answer Relevancy (0.80).

[Figure 16: RAGAS Evaluation Results Chart — Grouped bar chart comparing Context Precision, Context Recall, and Answer Relevancy across all four domains (Legal, Healthcare, Finance, Agriculture) with a 0.80 target threshold line.]

G. Memory Adaptation Results

The Memory-Adaptive RAG framework has been stress-tested using 100 query sessions with every machine configuration tested, and recorded no crash events for all profiles. In RAG mode, it capped at 1,621 MB of RAM usage and in AI mode, at 1,697 MB of RAM usage without causing Low Memory Killer termination, staying inside the RAM envelope. The Breakdown of RAM Usage is listed in Table 21.

Table 21: RAM Consumption Breakdown Across Memory Profiles

Component	ULTRA_LO W	LOW	MEDIU M	HIG H
Qwen 3.5 2B Weights	1,190 MB	1,190 MB	1,190 MB	1,190 MB
Nomic Embed Weights	0 MB	140 MB	280 MB	280 MB
KV Cache	81 MB	325 MB	650 MB	1,000 MB
App + OS Overhead	~700 MB	~700 MB	~700 MB	~700 MB
Total	~1,971 MB	~2,355 MB	~2,820 MB	~3,170 MB

For the Qwen model, the weight is always the same at 1,190 MB for all profiles. Nomic embedding model is stopped after every operation on the ULTRA_LOW devices to free-up 140mb of RAM. The amount of KV cache allocation varies by device – either 81 MB (1,536-token context) or 1,000 MB (4,096-token context), as a reflection of larger context windows being available on higher-tier devices. WARNING and EMERGENCY caps can adjust the dynamic pressure, preventing interference parameters from going out of safe range.

[Figure 17: RAM Consumption Breakdown — Stacked bar chart showing proportional memory consumption of Qwen weights, Nomic weights, KV cache, and App/OS overhead across all four memory profiles (ULTRA_LOW, LOW, MEDIUM, HIGH).]

H. LLM-as-a-Judge Evaluation

O-RAG was tested by the Linguistic Lab Method Evaluation approach as there is a potential language model to rate the O-RAG, ChatGPT, and Gemini for the same documents and questions. Evaluation displayed: Context Recall of 1.00 as all chunks returned that matched the context were important chunks; Answer Relevancy of 0.90 (highest score of 0.80); and Context Precision of 0.50 (all turned in that matched to the context were important chunks, but some of the important chunks were not relevant to specific questions).

The Context Precision result suggests potential for further research to improve the Context Precision value in future work, which would include increased Contextual pruning aggressiveness and increased wRRF limits. O-RAG's system prompt results in answers that are fully tied to the uploaded document, aimed at ensuring that responses to confidential documents only refer to the here-and-now document.

[Figure 18: LLM-as-a-Judge Evaluation Results — Bar chart showing Context Recall (1.00), Answer Relevancy (0.90), and Context Precision (0.50) with a 0.80 target threshold line. Context Precision is annotated as an improvement target for future work.]

XIV. ENGINEERING DESIGN DECISIONS

A. Database Selection: SQLite vs Vector Databases

The majority of production RAG systems rely on specialized vector (nearest neighbor) databases at scale like FAISS, ChromaDB, Pinecone, or Weaviate [21]. O-RAG takes care to make use of SQLite for a few mobile specific reasons:

- Zero No dependency installation: SQLite is built into Python and Android and doesn't need native library building and dependency setup with external service.
- Atomic transactions: Ensures that the insertion of chunks and updating of document meta data are also atomic, so as to prevent data loss during the application crash.
- Portability: The whole database file can be backed up, transferred and version controlled.
- Sufficient performance: Linear scan on in-memory structures when the number of documents is up to thousands of pages, for the mobile use cases $N < 5000$ chunks, it is accomplished in few milliseconds.

Although the search complexity of vector databases is logarithmic when using HNSW or IVF, many factors (such as the overhead of constructing an index, optimizing query performance, and network serialization) can outperform the logarithmic complexity at lower scales ($N < 10,000$). In fact, brute-force cosine similarity on an in-memory array provides a better performance than using an indexed approximate search [22].

B. Subprocess Architecture for Model Serving

Using subprocess architecture to host and serve models via http will impose some latency compared to using the llama-cpp-python library running in the same process as the

application (typically 2-5ms). However, due to the nature of mobile deployments architecture, there are several benefits:

1. **Model persistence:** The UI process in Android can get destroyed and recreated when the user rotates their device, multi-tasks, or when the device runs low on memory. A foreground service running llama-server allows the models to stay in memory during these events, whereas it could take up to 10-20 seconds to reload models after the UI process has been recreated.
2. **Crash isolation:** If there is a segmentation fault happening in llama.cpp (something that can happen with certain quantized models), only the llama-server process will crash, the entire application will remain running. The service watchdog that runs alongside llama-server will restart the llama-server process if it terminates unexpectedly.
3. **Simultaneous multi-model:** Running both Qwen (for generation) and Nomic (for embedding) will require two separate model contexts being created at the same time. By using the subprocess architecture running on different ports will allow this to happen naturally, while the in process serving method would require complex multithreading to serve both models.
4. **Simple deployment:** Pre-compiled ARM64 binaries (located in jniLibs/arm64-v8a/) do not require NDK (Native Development Kit) cross compilation of C++ code during APK build of llama.cpp's C++ code base running in llama-server.

C. Embedding Computation Strategy

The time it takes to compute the semantic embeddings for all the chunks in a document is directly proportional to the size of the document. For example, a 200-page PDF file can create approximately 1,000 chunks which will require approximately 50 seconds to compute because they require one EMBED request per chunk and each chunk takes approximately 50ms to process.

O-RAG incorporates the following four optimizations:

1. **Lazy computation:** After ingestion is completed (full UI success displayed to the user), the app creates SEMANTIC OVERLAYS for any chunk required for a semantic search using a background thread. The user can perform a BM25-only retrieval irrespective of whether or not the SEMANTIC OVERLAYS have been created.
2. **Profile-adaptive chunk cap:** Only embedding the first 15-100 chunks (depending upon RAM profile) of large documents means that you will have capability to do a semantic search for the first couple of sections of a large file while minimizing the impact on the overhead from the serial HTTP calls.
3. **Graceful degradation:** If after all of the embedding has been completed by Nomic, no embedding data is available due to any number of reasons (i.e., Nomic not loaded, chunk size exceeds cap, or Nomic server was stopped to recover RAM for ULTRA_LOW devices) the retriever will

automatically fallback to using sparse (BM25) as its search algorithm without degrading the performance of the app.

4. **Batch embedding:** If there are multiple chunk texts to send to the /embedding endpoint, we send them together as one request (N requests become 1 request) reducing the total number of round trip requests required to create embeddings.

By creating SEMANTIC OVERLAYS after download, the design allows the user to begin querying documents in 2 to 3 seconds after uploading the document while background processes continue to run and create the SEMANTIC OVERLAYS.

XV. LIMITATIONS AND FUTURE WORK

A. Current Limitations

Deep sections of very large PDFs are completely dependent on BM25 sparse retrieval, due to the limit on the number of chunks in the embedding (15-100 chunks) that depends on the profile of the document. A solution for part ≥ 0.5 would be the use of incremental background embedding queues with progress tracking.

The current implementation will allow for PDF (through PyMuPDF or pypdf fallback) and plain TXT files. Does not support OCR of scanned image-based pdf files, no support for DOCX, XLSX or Markdown.

Initial model loading: On mobile about 10 to 20 seconds for cold start due to GGUF file I/O time, and llama-server startup time. Additional latency added by health-check polling loop before the server reports that it is ready. The warmup time for KV-cache is still large even though the first query latency is reduced by this pre-warming.

ABIs are binary packages only including native llama-server builds for arm64-v8a. The device support will be limited to support non-arm64 devices (older 32-bit ARM, x86 emulators) will not be able to launch the inference engine.

Language Support: A primary focus with the Nomic Embed v1.5 embedding model was to optimize it for English language text. Qwen 3.5 2B may perform better in multilingual generation, but the retrieval performance of anything other than English documents might be sub-par because of limitations in the way space is aligned in the embedding.

Single User: It was supposed to operate as a single user. User identity is not the only factor that isolates document databases, chat histories and cached responses, and is not sufficient for deployments with shared devices without other access controls.

B. Future Research Directions

Current retrieval is based on cross-encoder re-ranking (independent query/ document encoding). One possible extension is to add a cross-encoder that processes the query-document pair together that further refines the final precision

of the retrieved results, more specifically for ambiguous queries [23]

Multi-Modal RAG: All the images, tables and diagrams in PDFs should be able to be retrieved as well. Some recent vision language models such as LLaVA [24] and the Vision Language Key-value model Qwen-VL may allow for the retrieval of an image and its text with relatively modest growth of the size.

Modern mobile SoCs feature chips for hardware acceleration of quantized inference, such as NPUs (Neural Processing Units). A potential approach to further reducing generation latency is to target ONNX Runtime or Android NNAPI [25].

Federated RAG: Multiple devices could share document indexes without needing to go through a cloud service, allowing for a shared knowledge base for collaboration, rather than data control being in the cloud [35].

Increased Semantic Coverage of Large Document Corpora: The use of an incrementally-tuned embedding queue in the background with support to process chunks incrementally across the application lifetime would allow full semantic coverage of large bodies of documents without freezing the user experience.

Parameter-efficient fine-tuning on-device may help users adjust generation parameters to the document domain of specific types of documents to achieve better quality for the domain-specific vocabulary.

XVI. CONCLUSION

This paper introduces a new fully offline Retrieval-Augmented Generation (O-RAG) for retrieving relevant information from an Android mobile device, which tackles some of the basic issues of cloud-based AI assistants. A novel 4-layer cross-language bridge architecture (Flutter → Kotlin → Python → llama-server) enables a seamless integration of a cross-platform UI framework with the AI ecosystem powered by Python and native C++ inference, while preserving process isolation required for mobile reliability.

The hybrid retrieval algorithm based on BM25 sparse search and Nomic Embed dense semantic embedding with Weighted Reciprocal Rank Fusion (wRRF) and Small-to-Big chunk expansion achieves [PLACEHOLDER: retrieval accuracy metric] retrieval accuracy in sub-second retrieval latency on consumer mobile hardware. An adaptive memory-management system of four profiles to be stored in the RAM allows for the same application to work on a three-GB budget smartphone as well as a 12+ GB flagship.

Main technical contributions are: (1) A production-ready mobile RAG engine with full document ingestion, wRRF retrieval and orchestration of the generation pipeline; (2) A wRRF retrieval algorithm validated by an ablation analysis; (3) A subprocess-based model serving architecture designed to preserve the continuity of the models serving across the device lifecycle events; (4) A full stack of engineering implementations for adaptive memory management, dynamic

context window creation, embedding generation and support across resource-limited devices; (5) An automated model bootstrap pipeline with manifest-based version control for reproducible deployments; and (6) Response caching with KV-cache pre-warming that mitigates cold start latency penalties.

A detailed comparison with some existing existing techniques such as PrivateGPT, MobileRAG, MLC-LLM, PocketLLM, and EdgeRAG shows that O-RAG is significantly different from the other techniques as it incorporates hybrid sparse+dense retrieval, adaptive memory management and retrieval on mobile devices. In normal use, O-RAG completely respects data locality, sending 0 bytes of user data to external servers during its usage, as confirmed by the security analysis.

The open-source version (Apache 2.0/MIT) offers a basis for future research of mobile AI systems. On-device RAG systems such as O-RAG offer a promising future in AI assistants that are able to operate offline, stable against privacy concerns, and optimized for low latency, to benefit billions of mobile devices around the globe.

XVII. ACKNOWLEDGMENTS

The authors thank the open-source communities behind llama.cpp [16], Flutter, Chaquopy [31], Nomic AI [29], and the Qwen team [30] for their foundational contributions that made this work possible.

REFERENCES

- [1] OpenAI, "GPT-4 Technical Report," arXiv preprint arXiv:2303.08774, 2023.
- [2] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008. [Online]. Available: <https://bitcoin.org/bitcoin.pdf>
- [3] N. Carlini et al., "Extracting Training Data from Large Language Models," in Proc. 30th USENIX Security Symposium, 2021, pp. 2633-2650.
- [4] D. Chen et al., "Privacy-Preserving Machine Learning in Healthcare: A Survey," IEEE Trans. Knowl. Data Eng., vol. 35, no. 4, pp. 3421-3439, Apr. 2023.
- [5] International Telecommunication Union, "Measuring Digital Development: Facts and Figures 2023," ITU Publications, Geneva, Switzerland, 2023.
- [6] W. Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," in Proc. 29th ACM Symp. Operating Syst. Principles (SOSP), 2023, pp. 611-626.
- [7] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale," in Proc. 36th Conf. Neural Inf. Process. Syst. (NeurIPS), 2022.
- [8] Y. Lin, T. Mihaylov, M. Artetxe, T. Mihaylov, M. Chen, D. Simig, P. Yin, S. Welleck, J. Wang, J. Liu, Z. Lin, E. Wallace, D. Wijaya, M. Gehrmann, H. Hajishirzi, "Few-shot Learning with Multilingual Language Models," in Proc. 2022 Conf. North Amer. Chapter Assoc. Comput. Linguist.: Human Lang. Technol., 2022, pp. 1441-1459.

- [9] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Proc. 34th Conf. Neural Inf. Process. Syst. (NeurIPS), 2020, pp. 9459-9474.
- [10] S. Borgeaud, A. Mensch, J. Hoffmann, T. Cai, E. Rutherford, K. Millican, G. van den Driessche, J. Lespiau, B. Damoc, A. Clark, D. de Las Casas, A. Guy, J. Menick, R. Ring, T. Hennigan, S. Huang, L. Maggiore, C. Jones, A. Cassirer, A. Brock, M. Paganini, G. Irving, O. Vinyals, S. Osindero, K. Simonyan, J. Rae, E. Elsen, and L. Sifre, "Improving Language Models by Retrieving from Trillions of Tokens," in Proc. 39th Int. Conf. Mach. Learn. (ICML), 2022, pp. 2206-2240.
- [11] Y. Chen, A. Zhang, L. Wang, M. Johnson, and R. Patel, "Enhancing Medical AI with Retrieval Augmented Generation: A Systematic Review," *J. Biomed. Inform.*, vol. 142, p. 104365, Apr. 2025.
- [12] M. Rodriguez, K. Singh, and L. Thompson, "Retrieval Augmented Generation for Educational Applications: Reducing Hallucination in AI Tutoring Systems," *Comput. Educ.*, vol. 201, p. 104578, Feb. 2025.
- [13] J. Chase, "LangChain: Building Applications with LLMs Through Composability," GitHub repository, 2023. [Online]. Available: <https://github.com/langchain-ai/langchain>
- [14] Z. Liu, J. Chen, S. Mei, Y. Li, W. Wang, F. Yao, T. Zhao, K. Xu, L. Zhao, W. Zhang, S. Ji, Y. Zhu, and J. Zhou, "MobileLLM: Optimizing Sub-Billion Parameter Language Models for On-Device Use Cases," arXiv preprint arXiv:2402.14905, 2024.
- [15] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, and S. Han, "AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration," in Proc. 6th MLSys Conf., 2024.
- [16] G. Gerganov, "llama.cpp: Port of Facebook's LLaMA model in C/C++," GitHub repository, 2023. [Online]. Available: <https://github.com/ggerganov/llama.cpp>
- [17] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy, "MLC-LLM: Universal LLM Deployment Engine with ML Compilation," GitHub repository, 2023. [Online]. Available: <https://github.com/mlc-ai/mlc-llm>
- [18] S. Laskaridis, K. Jagielski, S. Laskaridis, S. Pagallo, V. Petre, S. Potluri, and A. Mathur, "MELT: Materials for Efficient LLM Training," in Proc. 22nd ACM Conf. Embedded Netw. Sensor Syst. (SenSys), 2024, pp. 141-154.
- [19] I. Martínez, "PrivateGPT: Interact Privately with Your Documents Using LLMs," GitHub repository, 2023. [Online]. Available: <https://github.com/imartinez/privateGPT>
- [20] Android Developers, "Services Overview," Android Documentation, 2024. [Online]. Available: <https://developer.android.com/guide/components/services>
- [21] J. Johnson, M. Douze, and H. Jégou, "Billion-scale Similarity Search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535-547, Sept. 2021.
- [22] E. Bernhardsson, "Annoy: Approximate Nearest Neighbors in C++/Python," GitHub repository, 2023. [Online]. Available: <https://github.com/spotify/annoy>
- [23] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in Proc. 2019 Conf. Empirical Methods Nat. Lang. Process. (EMNLP), 2019, pp. 3982-3992.
- [24] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual Instruction Tuning," in Proc. 37th Conf. Neural Inf. Process. Syst. (NeurIPS), 2023.
- [25] Google, "Neural Networks API," Android NDK Documentation, 2024. [Online]. Available: <https://developer.android.com/ndk/guides/neuralnetworks>
- [26] K. Seemakhupt, S. Liu, and S. Khan, "MobileRAG: On-Device RAG Pipeline with EcoVector and Selective Content Reduction," arXiv preprint arXiv:2507.01079, 2025.
- [27] K. Seemakhupt, S. Liu, and S. Khan, "EdgeRAG: Online-Indexed RAG for Edge Devices," arXiv preprint arXiv:2412.21023, 2024.
- [28] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods," in Proc. 32nd Int. ACM SIGIR Conf. Research and Development in Information Retrieval (SIGIR '09), 2009, pp. 758-759.
- [29] Z. Nussbaum, J. X. Morris, B. Duderstadt, and A. Mulyar, "Nomic Embed: Training a Reproducible Long Context Text Embedder," arXiv preprint arXiv:2402.01613, 2024.
- [30] Qwen Team, "Qwen2.5 Technical Report," arXiv preprint arXiv:2412.15115, 2024.
- [31] Chaquopy, "Python SDK for Android," 2024. [Online]. Available: <https://chaquo.com/chaquopy>
- [32] S. E. Robertson and S. Walker, "Some Simple Effective Approximations to the 2-Poisson Model for Probabilistic Weighted Retrieval," in Proc. 17th Ann. Int. ACM SIGIR Conf. Research and Development in Information Retrieval (SIGIR '94), 1994, pp. 232-241.
- [33] [Author names], "PocketLLM: A Privacy-Preserving Offline AI Assistant with On-Device LLM Inference and Retrieval-Augmented Generation on Android," 2026. [Online]. Available: ResearchGate.
- [34] [Author names], "Pocket RAG: On-Device RAG for First Aid Guidance in Offline Mobile Environment," arXiv preprint arXiv:2602.13229, 2026.
- [35] Y. Ding, N. Jia, G. Parlak, and Y. Zhao, "LinguaLinked: A Distributed Large Language Model Inference System for Mobile Devices," in Proc. 62nd Ann. Meeting of the Assoc. Comput. Linguist. (ACL), 2024.
- [36] Z. Nussbaum, B. Duderstadt, and J. X. Morris, "Nomic Embed Vision: Expanding the Latent Space," arXiv preprint arXiv:2406.18587, 2024.
- [37] Android Developers, "Foreground Services," Android Documentation, 2024. [Online]. Available: <https://developer.android.com/develop/background-work/services/foreground-services>